ChatGPT

Go learning plan and unit-test exercise set for a .NET developer

Executive summary

This report lays out an accelerated, hands-on path for a .NET (C#) developer to get productive in Go fast—specifically to internalize Go syntax, the "Go way" (packages + composition + small interfaces), explicit error handling, and Go's built-in concurrency model (goroutines/channels + context cancellation). The plan is structured as a progressive curriculum where each topic is mapped to familiar .NET equivalents, with time estimates, learning objectives, recommended primary sources (Go official docs + Go blog + Tour of Go, plus Go by Example), and copy-pasteable Go unit-test exercises for deliberate practice. [1]

Core idea: you'll learn Go the way Go expects you to work: create a module (`go mod init`), write small packages, format with `gofmt`, test with `go test`, and use explicit errors instead of exceptions. [2]

Suggested pace (fast-track): ~25–40 focused hours (about 2–3 weeks at ~1–2 hours/day, or ~1 week full-time). Time estimates below assume you already know programming fundamentals and are mainly learning Go's syntax + idioms + toolchain. (These are estimates, not guarantees.)

Feature comparison and mental model mapping

High-signal comparison table

Topic	.NET / C# mental model	Go mental model	What this changes in how you code
Types + values	Reference vs value types; lots of "class by default" patterns	Values are central; structs + slices/maps are common; pointers exist but are explicit	You think in "data + functions" more than "class hierarchies". [3]
Error handling	Exceptions (`try/catch/finally`, `throw`)	Errors are values returned explicitly (`value, err`)	Call sites check errors; you add context with wrapping; panic/recover is not the default control flow. [4]
Generics	Mature generics in runtime + language	Added in Go 1.18 via type parameters; guidance emphasizes using them when they help, not everywhere	You'll see generics, but idiomatic Go still leans on interfaces + concrete types where that's clearer. [5]

Topic	.NET / C# mental model	Go mental model	What this changes in how you code
Concurrency	Threads/ ThreadPool, `Task`, TPL, `async/await`, cancellation tokens	goroutines + channels + `select`; memory model documented; `context.Context` is the standard cancellation/timeout mechanism	You'll reach for goroutines/channels and context-driven cancellation; race detector is a normal part of your workflow. [6]
Package management	NuGet + `dotnet restore`; project/ solution files	Modules (`go.mod`, `go.sum`), module paths + semantic import versioning	You manage dependencies via Go modules and the `go` command; module path is part of import paths. [7]
Testing	xUnit/NUnit/ MSTest frameworks, attributes	Built-in `testing` package + `go test`; table-driven tests are idiomatic	Tests are plain functions (`TestXxx`), and "table-driven" is the default style for many cases. [8]
Formatting + tooling	IDE rules vary; formatters exist but are optional culturally	`gofmt` is the standard; `go fmt`, `go vet`, `go doc`, `go fix` are first-class	Go codebases converge on one formatting style; tooling consistency is part of the culture. [9]

Practical mapping: Go concept ↔ .NET equivalent

Go concept	Closest .NET parallel	Key differences you must internalize
Package	Namespace + assembly-ish unit	Packages are directory-based; imports are by module path + subdir. [10]
Module (`go.mod`)	Solution/project + package references	Module path is identity; `go` tooling manages reproducible builds. [11]
Struct	`record` / `struct` / POCO	Methods exist, but no classes; composition dominates; embedding is common. [12]
Interface	`interface`	**Implicit** satisfaction (no `: IFoo`); method sets matter (value vs pointer). [13]

Go concept	Closest .NET parallel	Key differences you must internalize
`defer`	`using` / `try/finally`	Executes on function return (LIFO); used heavily for cleanup. [14]
goroutine	`Task` / threadpool work item	Very cheap to start; communicate with channels; still must manage lifetimes. [15]
Channel	`Channel<T>` / `BlockingCollection<T>` vibe	Built-in language feature; `select` multiplexes; critical for orchestration. [16]
`context.Context`	`CancellationToken` (+ deadlines)	Standard library idiom for cancellation/timeouts across goroutines. [17]
`errors.Is/As`, wrapping, `errors.Join`	Exception types + inner exceptions	Error "trees" via wrapping/unwrapping; inspect with `Is/As`; join multiple. [18]

Curriculum timeline and objectives

Timeline learning schedule

```
gantt
  title Go fast-track schedule for a .NET developer
  dateFormat  YYYY-MM-DD
  axisFormat  %b %d

  section Foundations
  Setup + workflow (install, go tool, module init)       :a1, 2026-04-06, 1d
  Basic syntax + control flow                            :a2, after a1, 2d
  Types (slices, maps, structs, pointers, zero values)   :a3, after a2, 2d

  section Core Go model
  Functions + methods                                    :b1, after a3, 1d
  Interfaces + method sets                               :b2, after b1, 2d
  Error handling + wrapping + defer/panic/recover         :b3, after b2, 2d

  section Concurrency + ecosystem
  Goroutines + channels + select + context               :c1, after b3, 3d
  Packages/modules + layout                              :c2, after c1, 1d
  Testing patterns + tooling (go test, gofmt, vet, race) :c3, after c2, 2d
```

```
  section Capstone
  Small project + tests                                    :d1, after c3, 2d
```

The schedule is intentionally front-loaded on Go's biggest "mental model deltas" from .NET: packages/modules, interfaces (implicit), explicit errors, and concurrency primitives. [19]

Curriculum table with time estimates and primary resources

Topic	Est. time	Learning objectives	Recommended primary resources
Setup	1–2h	Install Go, verify `go version`, create module, run tests	Install + getting started tutorial [20]; Create module tutorial [21]
Basic syntax	2–3h	`package/import`, declarations (`:=`, `var`, `const`), formatting expectations	Go Spec (reference) [22]; Tour of Go [23]; gofmt blog [24]
Types	4–6h	Primitives, structs, pointers, slices/maps, zero values	Tour (methods/types sections) [25]; Go "maps" blog (zero value patterns) [26]; Effective Go (style/zero value) [27]
Control flow	2–3h	`if`, `for`, `switch`, `range`, `defer` basics	Go Spec [22]; Tour flow control [23]
Functions	2–4h	Multiple returns, named returns, variadics, closures	Tour [28]; Go Spec [22]
Methods	2–3h	Receiver methods, pointer vs value receiver intuition	Tour: Methods [29]; Method sets wiki [30]
Interfaces	3–5h	Implicit implementation; interface design; type assertions	Tour: Interfaces [31]; Method sets wiki [30]; Code Review Comments [32]
Error handling	3–5h	`error` interface, wrapping/unwrapping, `errors.Is/As/Join`, `defer`, limited use of `panic`	Tour: Errors [33]; "Error handling and Go" [34]; "Errors are values" [35]; errors pkg docs [36]; "Defer, Panic, and Recover" [37]
Concurrency	6–10h	goroutines/channels/select; cancellation via context; memory model + race detector	Tour concurrency [38]; Pipelines blog [39]; Context blog [40]; Memory model [41]; Race detector article [42]
Packages/modules	3–5h	import paths, module layout, go.mod essentials, semantic import versioning	How to Write Go Code [43]; Modules ref [44]; go.mod ref [45]; Organizing a module [46]
Testing	3–5h	`testing` basics, table-driven tests, subtests	testing pkg [47]; TableDrivenTests wiki [48]; Go by Example testing [49]

Topic	Est. time	Learning objectives	Recommended primary resources
Tooling	2–4h	`go test`, `go fmt/gofmt`, `go doc`, `go vet`, `go fix`, `-race`	Command docs [50]; gofmt blog [24]; go fix blog [51]; race detector article [42]
Idiomatic patterns	4–8h	small interfaces, useful zero values, composition/embedding, "Go proverbs" as heuristics	Effective Go (with its caveat) [52]; Code Review Comments [32]; "Share Memory By Communicating" [53]; maps blog [26]

Topic packs with .NET parallels, code diffs, and unit-test exercises

Everything below is "progressive": later topics assume you can already run `go test` and are comfortable reading Go code. Official docs heavily emphasize learning by doing (Tour + tutorials), and Go tooling is designed around that workflow. [54]

Setup

Concise explanation: Setup in Go is basically "install Go, verify `go version`, create a module, run code with `go run`, and run tests with `go test`." The official getting-started tutorials explicitly guide you through install → hello world → using the `go` command and modules. [55]

.NET parallel: `dotnet --info`, `dotnet new`, `dotnet run`, `dotnet test` vs `go version`, `go mod init`, `go run`, `go test`. [56]

Side-by-side (commands)

```
# .NET
dotnet new console -n HelloApp
cd HelloApp
dotnet run
dotnet test
```

```
# Go
mkdir helloapp
cd helloapp
go mod init example.com/helloapp
go run .
go test ./...
```

Unit-test exercises (copy-paste into `setup_exercises_test.go`)

```go
package learn

import (
    "strings"
    "testing"
)

// EXERCISE 1: Implement setupHello.
// - If name is empty/whitespace => "Hello, world!"
// - Else => "Hello, <name>!"
func TestSetupHello(t *testing.T) {
    tests := []struct {
        name string
        in   string
        want string
    }{
        {"normal", "Paul", "Hello, Paul!"},
        {"empty", "", "Hello, world!"},
        {"spaces", "   ", "Hello, world!"},
    }

    for _, tc := range tests {
        t.Run(tc.name, func(t *testing.T) {
            got := setupHello(tc.in)
            if got != tc.want {
                t.Fatalf("setupHello(%q) = %q; want %q", tc.in, got, tc.want)
            }
        })
    }
}

// EXERCISE 2: Implement setupParseGoVersion.
// Parse "go version go1.22.3 darwin/arm64" or "go1.22.3".
// Return major=1, minor=22 for that example.
// Return ok=false if you can't parse.
func TestSetupParseGoVersion(t *testing.T) {
    tests := []struct {
        in         string
        wantMajor  int
        wantMinor  int
        wantParsed bool
    }{
        {"go version go1.22.3 darwin/arm64", 1, 22, true},
        {"go1.21.0", 1, 21, true},
        {"go version deez nuts", 0, 0, false},
    }
```

```
    for _, tc := range tests {
        t.Run(tc.in, func(t *testing.T) {
            maj, min, ok := setupParseGoVersion(tc.in)
            if ok != tc.wantParsed {
                t.Fatalf("setupParseGoVersion(%q) ok=%v; want %v", tc.in, ok,
tc.wantParsed)
            }
            if ok && (maj != tc.wantMajor || min != tc.wantMinor) {
                t.Fatalf("setupParseGoVersion(%q)=(%d,%d); want (%d,%d)",
tc.in, maj, min, tc.wantMajor, tc.wantMinor)
            }
        })
    }
}

// EXERCISE 3: Implement setupModulePathFromGoMod.
// From a go.mod file text, extract module path from the "module <path>" line.
func TestSetupModulePathFromGoMod(t *testing.T) {
    goMod := strings.TrimSpace(`
module example.com/mymodule

go 1.22

require (
    example.com/dep v1.2.3
)
`)
    if got := setupModulePathFromGoMod(goMod); got != "example.com/mymodule" {
        t.Fatalf("setupModulePathFromGoMod(...)=%q; want %q", got, "example.com/
mymodule")
    }
}

func setupHello(name string) string {
    // TODO: implement
    return ""
}

func setupParseGoVersion(s string) (major int, minor int, ok bool) {
    // TODO: implement
    return 0, 0, false
}

func setupModulePathFromGoMod(goModText string) string {
    // TODO: implement
    return ""
}
```

Basic syntax

Concise explanation: Go syntax is intentionally small: packages/imports, `func`, `var/const`, short declarations (`:=`), and control flow forms that avoid parentheses around conditions. Formatting is standardized via `gofmt`, and the toolchain is designed so most code looks structurally similar across projects. [57]

.NET parallel: you'll miss some "ceremony" (no `class Program`, no `public static void Main(string[] args)` signature pressure), but gain very direct, file-level structure. [58]

Side-by-side (hello world)

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Hello, world!");
    }
}
```

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, world!")
}
```

Unit-test exercises (copy-paste into `syntax_exercises_test.go`)

```
package learn

import (
    "strings"
    "testing"
)

// EXERCISE 1: Implement syntaxDefaultString.
// Return def if s is empty or whitespace; otherwise return s trimmed.
func TestSyntaxDefaultString(t *testing.T) {
    if got := syntaxDefaultString("   ", "fallback"); got != "fallback" {
        t.Fatalf("got %q; want %q", got, "fallback")
```

```
    }
    if got := syntaxDefaultString("  hi  ", "fallback"); got != "hi" {
        t.Fatalf("got %q; want %q", got, "hi")
    }
}

// EXERCISE 2: Implement syntaxJoinWithComma.
// Join parts with ", ".
// - nil or empty slice => ""
// - Any element should be trimmed.
func TestSyntaxJoinWithComma(t *testing.T) {
    if got := syntaxJoinWithComma(nil); got != "" {
        t.Fatalf("got %q; want empty", got)
    }
    if got := syntaxJoinWithComma([]string{" a", "b ", "  c  "}); got != "a, b,
c" {
        t.Fatalf("got %q; want %q", got, "a, b, c")
    }
}

// EXERCISE 3: Implement syntaxHasPrefixInsensitive.
// Case-insensitive prefix check (ASCII-lower is fine for this exercise).
func TestSyntaxHasPrefixInsensitive(t *testing.T) {
    if !syntaxHasPrefixInsensitive("Golang", "go") {
        t.Fatalf("expected true")
    }
    if syntaxHasPrefixInsensitive("Golang", "lang") {
        t.Fatalf("expected false")
    }
}

func syntaxDefaultString(s, def string) string {
    // TODO: implement
    return ""
}

func syntaxJoinWithComma(parts []string) string {
    // TODO: implement (hint: strings.Builder can be handy)
    return ""
}

func syntaxHasPrefixInsensitive(s, prefix string) bool {
    // TODO: implement (hint: strings.ToLower + strings.HasPrefix)
    _ = strings.Builder{} // remove after implementing if unused
    return false
}
```

Types

Concise explanation: Go is statically typed (spec-defined), and most day-to-day data modeling uses structs plus slices/maps. The "zero value" concept matters more than in many .NET codebases: nil slices can often be used safely (e.g., append), whereas nil maps require initialization before assignment; blog posts, docs, and idioms leverage this heavily. [59]

.NET parallel: `List<T>` ↔ `[]T` (slice), `Dictionary<TKey,TValue>` ↔ `map[K]V`, `record/struct` ↔ `struct`. [60]

Side-by-side (collections + struct/record vibe)

```
var scores = new Dictionary<string,int> { ["a"] = 1, ["b"] = 2 };
var list = new List<int> { 1, 2, 3 };

record Person(string Name, int Age);
var p = new Person("Paul", 30);
```

```
scores := map[string]int{"a": 1, "b": 2}
list := []int{1, 2, 3}

type Person struct {
    Name string
    Age  int
}
p := Person{Name: "Paul", Age: 30}
```

Unit-test exercises (copy-paste into `types_exercises_test.go`)

```
package learn

import (
    "reflect"
    "sort"
    "testing"
)

type typesPerson struct {
    Name string
    Age  int
}

// EXERCISE 1: Implement typesSumInts.
// Must handle nil slices safely.
```

```go
func TestTypesSumInts(t *testing.T) {
    if got := typesSumInts(nil); got != 0 {
        t.Fatalf("got %d; want 0", got)
    }
    if got := typesSumInts([]int{1, 2, 3}); got != 6 {
        t.Fatalf("got %d; want 6", got)
    }
}

// EXERCISE 2: Implement typesInvertMap.
// Input: map[string]int{"a":1,"b":1,"c":2}
// Output: map[int][]string{1:["a","b"], 2:["c"]} (values sorted for test
determinism)
func TestTypesInvertMap(t *testing.T) {
    in := map[string]int{"b": 1, "a": 1, "c": 2}
    got := typesInvertMap(in)

    for _, v := range got {
        sort.Strings(v)
    }

    want := map[int][]string{1: {"a", "b"}, 2: {"c"}}
    if !reflect.DeepEqual(got, want) {
        t.Fatalf("got %#v; want %#v", got, want)
    }
}

// EXERCISE 3: Implement typesGroupByAgeDecade.
// Example: Age 29 -> "20s", Age 30 -> "30s".
func TestTypesGroupByAgeDecade(t *testing.T) {
    people := []typesPerson{
        {Name: "A", Age: 29},
        {Name: "B", Age: 30},
        {Name: "C", Age: 31},
    }
    got := typesGroupByAgeDecade(people)
    for _, v := range got {
        sort.Strings(v)
    }

    want := map[string][]string{
        "20s": {"A"},
        "30s": {"B", "C"},
    }
    if !reflect.DeepEqual(got, want) {
        t.Fatalf("got %#v; want %#v", got, want)
    }
}
```

```go
func typesSumInts(nums []int) int {
    // TODO: implement
    return 0
}

func typesInvertMap(in map[string]int) map[int][]string {
    // TODO: implement (hint: remember nil maps can't be assigned into)
    return nil
}

func typesGroupByAgeDecade(people []typesPerson) map[string][]string {
    // TODO: implement
    return nil
}
```

Control flow

Concise explanation: Go keeps control flow small: `if`, `for` (the only loop keyword), and `switch` (including type switches with interfaces). The Tour and spec define these constructs; idiomatic Go often prefers simple loops over heavy abstraction if it improves readability. [61]

.NET parallel: `foreach` ↔ `for range`, `while` ↔ `for`, `switch` exists but also powers concurrency (`select`) later. [62]

Side-by-side (loop + switch)

```csharp
foreach (var x in xs)
{
    if (x % 2 == 0) continue;
    sum += x;
}

switch (kind)
{
    case "A": return 1;
    default: return 0;
}
```

```go
for _, x := range xs {
    if x%2 == 0 {
        continue
    }
    sum += x
}
```

```
switch kind {
case "A":
    return 1
default:
    return 0
}
```

Unit-test exercises (copy-paste into `flow_exercises_test.go`)

```
package learn

import "testing"

// EXERCISE 1: Implement flowGCD using Euclid's algorithm.
// gcd(54, 24) == 6
func TestFlowGCD(t *testing.T) {
    if got := flowGCD(54, 24); got != 6 {
        t.Fatalf("got %d; want 6", got)
    }
    if got := flowGCD(0, 5); got != 5 {
        t.Fatalf("got %d; want 5", got)
    }
}

// EXERCISE 2: Implement flowFizzBuzz.
// Return a single string like "1,2,Fizz,4,Buzz" for n=5.
func TestFlowFizzBuzz(t *testing.T) {
    if got := flowFizzBuzz(5); got != "1,2,Fizz,4,Buzz" {
        t.Fatalf("got %q; want %q", got, "1,2,Fizz,4,Buzz")
    }
}

// EXERCISE 3: Implement flowFirstEven.
// Return (value,true) if any even exists; otherwise (0,false).
func TestFlowFirstEven(t *testing.T) {
    if v, ok := flowFirstEven([]int{1, 3, 4, 6}); !ok || v != 4 {
        t.Fatalf("got (%d,%v); want (4,true)", v, ok)
    }
    if v, ok := flowFirstEven([]int{1, 3, 5}); ok || v != 0 {
        t.Fatalf("got (%d,%v); want (0,false)", v, ok)
    }
}

func flowGCD(a, b int) int {
    // TODO: implement
```

```
    return 0
}

func flowFizzBuzz(n int) string {
    // TODO: implement
    return ""
}

func flowFirstEven(nums []int) (int, bool) {
    // TODO: implement
    return 0, false
}
```

Functions

Concise explanation: Functions are where Go starts feeling "different" if you're used to C# exceptions: multiple return values are normal (especially `(value, error)`), variadic functions are common, and closures are used but typically not to build giant expression pipelines. [63]

.NET parallel: multiple returns often become tuples, `out` parameters, or exceptions in C#. In Go they're simply return values. [64]

Side-by-side (tuple vs multi-return)

```
(int q, int r) DivMod(int a, int b) => (a / b, a % b);
```

```
func divMod(a, b int) (q, r int) { return a / b, a % b }
```

Unit-test exercises (copy-paste into `functions_exercises_test.go`)

```
package learn

import (
    "errors"
    "testing"
)

// EXERCISE 1: Implement funcsDivMod.
// Return error on b==0. Otherwise return q=a/b and r=a%b.
func TestFuncsDivMod(t *testing.T) {
    _, _, err := funcsDivMod(10, 0)
    if err == nil {
        t.Fatalf("expected error")
    }
```

```
    q, r, err := funcsDivMod(10, 3)
    if err != nil || q != 3 || r != 1 {
        t.Fatalf("got (q=%d,r=%d,err=%v); want (3,1,nil)", q, r, err)
    }
}

// EXERCISE 2: Implement funcsSumVariadic.
func TestFuncsSumVariadic(t *testing.T) {
    if got := funcsSumVariadic(); got != 0 {
        t.Fatalf("got %d; want 0", got)
    }
    if got := funcsSumVariadic(1, 2, 3); got != 6 {
        t.Fatalf("got %d; want 6", got)
    }
}

// EXERCISE 3: Implement funcsMakeCounter closure.
// It should return a function that yields 1,2,3,... on successive calls.
func TestFuncsMakeCounter(t *testing.T) {
    next := funcsMakeCounter()
    if next() != 1 || next() != 2 || next() != 3 {
        t.Fatalf("counter did not increment as expected")
    }
}

var errDivideByZero = errors.New("divide by zero")

func funcsDivMod(a, b int) (q int, r int, err error) {
    // TODO: implement
    return 0, 0, nil
}

func funcsSumVariadic(nums ...int) int {
    // TODO: implement
    return 0
}

func funcsMakeCounter() func() int {
    // TODO: implement
    return func() int { return 0 }
}
```

Methods

Concise explanation: In Go, a “method” is just a function with a receiver. Receivers can be value or pointer receivers, and method sets determine which interfaces a type implements—this is a real gotcha when coming from C# where reference semantics and interface implementation are explicit. [65]

.NET parallel: instance methods on classes/structs exist; but Go has no classes, and pointer vs value receiver is a design choice you make explicitly. [65]

Side-by-side (class vs struct+receiver)

```
class Counter {
    private int _n;
    public int Inc() => ++_n;
}
```

```
type Counter struct{ n int }

func (c *Counter) Inc() int { c.n++; return c.n }
```

Unit-test exercises (copy-paste into `methods_exercises_test.go`)

```
package learn

import "testing"

// EXERCISE 1-3: Implement methodsStack with methods:
// - Push(v int)
// - Pop() (int, bool)
// - Len() int
//
// Requirements:
// - Zero value must be usable: var s methodsStack; s.Push(1) should work.
// - Pop on empty => (0,false).
type methodsStack struct {
    // TODO: choose fields
}

func TestMethodsStackPushPop(t *testing.T) {
    var s methodsStack // zero value
    s.Push(10)
    s.Push(20)

    if s.Len() != 2 {
        t.Fatalf("Len()=%d; want 2", s.Len())
    }

    v, ok := s.Pop()
    if !ok || v != 20 {
        t.Fatalf("Pop()=(%d,%v); want (20,true)", v, ok)
```

```
    }
    v, ok = s.Pop()
    if !ok || v != 10 {
        t.Fatalf("Pop()=(%d,%v); want (10,true)", v, ok)
    }
    v, ok = s.Pop()
    if ok || v != 0 {
        t.Fatalf("Pop()=(%d,%v); want (0,false)", v, ok)
    }
}

func TestMethodsStackLenEmpty(t *testing.T) {
    var s methodsStack
    if s.Len() != 0 {
        t.Fatalf("Len()=%d; want 0", s.Len())
    }
}

func TestMethodsStackInterleave(t *testing.T) {
    var s methodsStack
    s.Push(1)
    s.Push(2)
    _, _ = s.Pop()
    s.Push(3)

    v, ok := s.Pop()
    if !ok || v != 3 {
        t.Fatalf("got (%d,%v); want (3,true)", v, ok)
    }
    v, ok = s.Pop()
    if !ok || v != 1 {
        t.Fatalf("got (%d,%v); want (1,true)", v, ok)
    }
}

// TODO: implement methods on *methodsStack / methodsStack:
// func (s *methodsStack) Push(v int) { ... }
// func (s *methodsStack) Pop() (int, bool) { ... }
// func (s methodsStack) Len() int { ... }
```

Interfaces

Concise explanation: Interfaces in Go are satisfied implicitly: if a type has the methods, it implements the interface—no declaration required. This supports “accept interfaces, return concrete” and encourages small interfaces, but it also means you must understand method sets (value vs pointer receiver) and how nil behaves with interfaces. [66]

.NET parallel: interface implementation is explicit (`class X : IFoo`), and the runtime is nominal; in Go it's structural at compile time. [67]

Side-by-side (explicit vs implicit)

```
interface IShape { double Area(); }
class Square : IShape { public double Side; public double Area() => Side *
Side; }
```

```
type shape interface{ Area() float64 }

type square struct{ side float64 }
func (s square) Area() float64 { return s.side * s.side } // implements shape
implicitly
```

Unit-test exercises (copy-paste into `interfaces_exercises_test.go`)

```
package learn

import (
    "io"
    "strings"
    "testing"
)

type interfacesShape interface {
    Area() float64
}

type interfacesSquare struct{ Side float64 }
type interfacesCircle struct{ Radius float64 }

// EXERCISE 1: Add Area() methods so both types implement interfacesShape.

func TestInterfacesTotalArea(t *testing.T) {
    shapes := []interfacesShape{
        interfacesSquare{Side: 2},  // area 4
        interfacesCircle{Radius: 1}, // area ~3.14159
    }
    got := interfacesTotalArea(shapes)
    if got < 7.1 || got > 7.2 {
        t.Fatalf("total area=%v; expected about 7.14..", got)
    }
}
```

```go
// EXERCISE 2: Implement interfacesDescribeAny using a type switch.
// Rules:
// - int => "int:<n>"
// - string => "string:<value>"
// - default => "unknown"
func TestInterfacesDescribeAny(t *testing.T) {
    if got := interfacesDescribeAny(7); got != "int:7" {
        t.Fatalf("got %q", got)
    }
    if got := interfacesDescribeAny("hi"); got != "string:hi" {
        t.Fatalf("got %q", got)
    }
    if got := interfacesDescribeAny(true); got != "unknown" {
        t.Fatalf("got %q", got)
    }
}

// EXERCISE 3: Implement interfacesReadAllUpper.
// Read everything from r, return the uppercase string.
func TestInterfacesReadAllUpper(t *testing.T) {
    got, err := interfacesReadAllUpper(strings.NewReader("go rocks"))
    if err != nil {
        t.Fatalf("unexpected err: %v", err)
    }
    if got != "GO ROCKS" {
        t.Fatalf("got %q; want %q", got, "GO ROCKS")
    }
}

func interfacesTotalArea(shapes []interfacesShape) float64 {
    // TODO: implement
    return 0
}

func interfacesDescribeAny(v any) string {
    // TODO: implement type switch
    return ""
}

func interfacesReadAllUpper(r io.Reader) (string, error) {
    // TODO: implement
    return "", nil
}

// TODO: implement Area methods:
// func (s interfacesSquare) Area() float64 { ... }
// func (c interfacesCircle) Area() float64 { ... }
```

Error handling

Concise explanation: In Go, errors are values and are returned explicitly. Official guidance emphasizes adding context, wrapping errors (so callers can inspect causes), and treating error handling as normal control flow—not as a rare exceptional path. `errors.Is` / `errors.As` traverse an error "tree" created via wrapping (including joined errors), and `errors.Join` standardizes multi-error aggregation. [68]

.NET parallel: C# primarily uses exceptions; they bubble up the call stack until caught. In Go, you check `err != nil`, return early, and callers decide what to do. [69]

Side-by-side (exceptions vs explicit error)

```
try
{
    var text = File.ReadAllText(path);
    return text;
}
catch (IOException ex)
{
    throw new InvalidOperationException($"read failed: {path}", ex);
}
```

```
b, err := os.ReadFile(path)
if err != nil {
    return "", fmt.Errorf("read failed %q: %w", path, err)
}
return string(b), nil
```

Unit-test exercises (copy-paste into `errors_exercises_test.go`)

```
package learn

import (
    "errors"
    "fmt"
    "strconv"
    "testing"
)

var errBadInput = errors.New("bad input")

type errorsFieldError struct {
    Field string
    Msg   string
```

```
}

func (e errorsFieldError) Error() string {
    return fmt.Sprintf("%s: %s", e.Field, e.Msg)
}

// EXERCISE 1: Implement errorsParseIntWithContext.
// Must wrap parsing errors with fmt.Errorf and %w.
func TestErrorsParseIntWithContext(t *testing.T) {
    _, err := errorsParseIntWithContext("nope")
    if err == nil {
        t.Fatalf("expected error")
    }
    // It should include the input in the message.
    if got := err.Error(); !containsAll(got, "nope") {
        t.Fatalf("error message %q should mention input", got)
    }
}

// EXERCISE 2: Implement errorsValidateAge.
// Rules:
// - age < 0 => errorsFieldError{Field:"age", Msg:"must be >= 0"}
// - age > 150 => errorsFieldError{Field:"age", Msg:"must be <= 150"}
// - else nil
func TestErrorsValidateAge(t *testing.T) {
    if err := errorsValidateAge(30); err != nil {
        t.Fatalf("unexpected: %v", err)
    }
    if err := errorsValidateAge(-1); err == nil || err.Error() != "age: must be
>= 0" {
        t.Fatalf("got %v", err)
    }
}

// EXERCISE 3: Implement errorsIsBadInput using errors.Is.
func TestErrorsIsBadInput(t *testing.T) {
    wrapped := fmt.Errorf("oops: %w", errBadInput)
    if !errorsIsBadInput(wrapped) {
        t.Fatalf("expected true")
    }
    if errorsIsBadInput(errors.New("other")) {
        t.Fatalf("expected false")
    }
}

// EXERCISE 4: Implement errorsJoinAll using errors.Join.
// Must discard nils and return nil if all are nil.
func TestErrorsJoinAll(t *testing.T) {
```

```
    if err := errorsJoinAll(nil, nil); err != nil {
        t.Fatalf("expected nil, got %v", err)
    }
    e1 := errors.New("e1")
    e2 := errors.New("e2")
    err := errorsJoinAll(e1, nil, e2)
    if err == nil {
        t.Fatalf("expected non-nil")
    }
    if !errors.Is(err, e1) || !errors.Is(err, e2) {
        t.Fatalf("joined error should match components via errors.Is")
    }
}

func errorsParseIntWithContext(s string) (int, error) {
    // TODO: implement; use strconv.Atoi and wrap parse errors
    _, _ = strconv.Atoi("0")
    return 0, nil
}

func errorsValidateAge(age int) error {
    // TODO: implement
    return nil
}

func errorsIsBadInput(err error) bool {
    // TODO: implement
    return false
}

func errorsJoinAll(errs ...error) error {
    // TODO: implement
    return nil
}

func containsAll(s string, parts ...string) bool {
    for _, p := range parts {
        if !stringsContains(s, p) {
            return false
        }
    }
    return true
}

// Keep this helper tiny so you don't need extra imports in the exercise.
func stringsContains(s, sub string) bool {
    return len(sub) == 0 || (len(s) >= len(sub) && (func() bool {
        // naive contains, good enough for unit tests
```

```
        for i := 0; i+len(sub) <= len(s); i++ {
            if s[i:i+len(sub)] == sub {
                return true
            }
        }
        return false
    }())
}
```

Concurrency

Concise explanation: Go concurrency is built around goroutines (cheap concurrent execution) and channels (typed communication), with `select` to wait on multiple channel operations. `context.Context` is the standard cancellation/deadline mechanism across goroutines. The Go memory model specifies what "synchronization" means, and the race detector (`go test -race`) is an expected tool for verifying concurrent code. [70]

.NET parallel: `Task`/TPL and `CancellationToken` map conceptually, but Go's defaults are different: you frequently compose goroutines + channels directly, and you must choose a cancellation story (usually `context`) early to avoid goroutine leaks. [71]

Side-by-side (TPL-ish vs goroutines/channels)

```
var tasks = items.Select(x => Task.Run(() => x * x)).ToArray();
var results = await Task.WhenAll(tasks);
```

```
jobs := make(chan int)
results := make(chan int)

go func() {
    for _, x := range items { jobs <- x }
    close(jobs)
}()

for i := 0; i < workers; i++ {
    go func() {
        for x := range jobs { results <- x * x }
    }()
}
```

Unit-test exercises (copy-paste into `concurrency_exercises_test.go`)

```go
package learn

import (
    "context"
    "errors"
    "sync"
    "testing"
    "time"
)

// EXERCISE 1: Implement concurrencySquareAll.
// Must process nums using at most `workers` goroutines concurrently.
// Must preserve input order in output.
// Return error if workers <= 0.
func TestConcurrencySquareAll(t *testing.T) {
    got, err := concurrencySquareAll([]int{1, 2, 3, 4}, 2)
    if err != nil {
        t.Fatalf("unexpected err: %v", err)
    }
    want := []int{1, 4, 9, 16}
    if !equalIntSlices(got, want) {
        t.Fatalf("got %v; want %v", got, want)
    }
    _, err = concurrencySquareAll([]int{1}, 0)
    if err == nil {
        t.Fatalf("expected error for workers<=0")
    }
}

// EXERCISE 2: Implement concurrencySelectFirstString.
// Read the first available value from a or b (use select).
func TestConcurrencySelectFirstString(t *testing.T) {
    a := make(chan string, 1)
    b := make(chan string, 1)
    b <- "B"
    got := concurrencySelectFirstString(a, b)
    if got != "B" {
        t.Fatalf("got %q; want %q", got, "B")
    }
}

// EXERCISE 3: Implement concurrencyCounterOwnedByGoroutine.
// Return three funcs: inc(), value(), stop().
// - inc increments and returns the new value
// - value returns current value
// - stop terminates the goroutine (idempotent)
func TestConcurrencyCounterOwnedByGoroutine(t *testing.T) {
```

```
    inc, val, stop := concurrencyCounterOwnedByGoroutine()
    defer stop()

    if val() != 0 {
        t.Fatalf("want 0")
    }
    if inc() != 1 || inc() != 2 {
        t.Fatalf("counter not incrementing")
    }
    if val() != 2 {
        t.Fatalf("want 2")
    }
}

var errWorkers = errors.New("workers must be > 0")

func concurrencySquareAll(nums []int, workers int) ([]int, error) {
    // TODO: implement worker pool with goroutines + channels
    // Hint: send (index,value) jobs; gather (index,result) and write into
output slice.
    return nil, nil
}

func concurrencySelectFirstString(a, b <-chan string) string {
    // TODO: implement using select
    return ""
}

func concurrencyCounterOwnedByGoroutine() (inc func() int, value func() int,
stop func()) {
    // TODO: implement using one goroutine that owns the state and channels for
commands.
    // Use a sync.Once or channel close discipline so stop() is safe to call
multiple times.
    return func() int { return 0 }, func() int { return 0 }, func() {}
}

func equalIntSlices(a, b []int) bool {
    if len(a) != len(b) {
        return false
    }
    for i := range a {
        if a[i] != b[i] {
            return false
        }
    }
    return true
}
```

```
// Optional pattern you can adopt after implementing: testing with timeouts to
avoid hangs.
func withTimeout(t *testing.T, d time.Duration) (context.Context,
context.CancelFunc) {
    t.Helper()
    return context.WithTimeout(context.Background(), d)
}

var _ = sync.Once{} // remove if unused after implementing
```

Packages and modules

Concise explanation: Go code is organized into packages (directories) and modules (a versioned set of packages rooted at a `go.mod`). Import paths are module path + subdirectory. Official docs also provide pragmatic layout guidance (`internal/` for non-exported packages, `cmd/` for commands) and detail module rules like semantic import versioning (major versions in import paths). [72]

.NET parallel: Think of "module" ≈ repo/package unit and "package" ≈ namespace+folder unit, but the import path is a real identifier in the tooling, not just a decoupled namespace name. [73]

Side-by-side (dependency declaration)

```
<!-- .csproj -->
<ItemGroup>
  <PackageReference Include="Example.Dep" Version="1.2.3" />
</ItemGroup>
```

```
// go.mod
module example.com/mymodule

go 1.22

require example.com/dep v1.2.3
```

Unit-test exercises (copy-paste into `modules_exercises_test.go`)

```
package learn

import "testing"

// EXERCISE 1: Implement modulesJoinImportPath.
// modulesJoinImportPath("example.com/m", "auth/token") => "example.com/m/auth/
token"
```

```
// Handle empty subdir => modulePath.
func TestModulesJoinImportPath(t *testing.T) {
    if got := modulesJoinImportPath("example.com/m", ""); got != "example.com/
m" {
        t.Fatalf("got %q", got)
    }
    if got := modulesJoinImportPath("example.com/m", "auth/token"); got !=
"example.com/m/auth/token" {
        t.Fatalf("got %q", got)
    }
}

// EXERCISE 2: Implement modulesWithMajorSuffix.
// If major >= 2, suffix "/v<major>" must be present.
// modulesWithMajorSuffix("example.com/m", 1) => "example.com/m"
// modulesWithMajorSuffix("example.com/m", 2) => "example.com/m/v2"
func TestModulesWithMajorSuffix(t *testing.T) {
    if got := modulesWithMajorSuffix("example.com/m", 1); got != "example.com/
m" {
        t.Fatalf("got %q", got)
    }
    if got := modulesWithMajorSuffix("example.com/m", 2); got != "example.com/m/
v2" {
        t.Fatalf("got %q", got)
    }
}

// EXERCISE 3: Implement modulesParseRequireLine.
// Example input: "require example.com/dep v1.2.3"
// Return mod="example.com/dep", ver="v1.2.3", ok=true.
func TestModulesParseRequireLine(t *testing.T) {
    mod, ver, ok := modulesParseRequireLine("require example.com/dep v1.2.3")
    if !ok || mod != "example.com/dep" || ver != "v1.2.3" {
        t.Fatalf("got (%q,%q,%v)", mod, ver, ok)
    }
    _, _, ok = modulesParseRequireLine("nonsense")
    if ok {
        t.Fatalf("expected ok=false")
    }
}

func modulesJoinImportPath(modulePath, subdir string) string {
    // TODO: implement
    return ""
}

func modulesWithMajorSuffix(modulePath string, major int) string {
    // TODO: implement
```

```
    return ""
}

func modulesParseRequireLine(line string) (mod string, version string, ok bool)
{
    // TODO: implement (basic split is fine for this exercise)
    return "", "", false
}
```

Testing

Concise explanation: Go's standard library `testing` package plus the `go test` command gives you a full baseline testing workflow. The idiomatic style is often table-driven tests (a slice of test cases iterated in a loop) plus subtests (`t.Run`). This is documented in official docs/wiki and reinforced by community education materials like Go by Example. [8]

.NET parallel: xUnit attributes vs Go's naming conventions (`TestXxx`) and explicit `t.Fatalf`/`t.Errorf`. [74]

Side-by-side (test skeleton)

```
// xUnit
[Fact]
public void Adds() => Assert.Equal(3, Add(1,2));
```

```
func TestAdd(t *testing.T) {
    if got := add(1, 2); got != 3 {
        t.Fatalf("got %d; want 3", got)
    }
}
```

Unit-test exercises (copy-paste into `testing_exercises_test.go`)

```
package learn

import (
    "strings"
    "testing"
)

// This file is intentionally table-driven + uses subtests,
// so you get used to idiomatic Go testing structures.
// EXERCISES: implement the functions under test.
```

```go
// EXERCISE 1: Implement testingNormalizeWhitespace.
// Convert any run of whitespace to a single space, and trim ends.
func TestTestingNormalizeWhitespace(t *testing.T) {
    tests := []struct {
        name string
        in   string
        want string
    }{
        {"already clean", "a b", "a b"},
        {"tabs/newlines", "a\tb\nc", "a b c"},
        {"leading/trailing", "  a   b  ", "a b"},
    }
    for _, tc := range tests {
        t.Run(tc.name, func(t *testing.T) {
            if got := testingNormalizeWhitespace(tc.in); got != tc.want {
                t.Fatalf("got %q; want %q", got, tc.want)
            }
        })
    }
}

// EXERCISE 2: Implement testingIsPalindromeASCII.
// Only consider letters/digits; ignore case.
// "A man, a plan, a canal: Panama" => true (ASCII-only handling is fine).
func TestTestingIsPalindromeASCII(t *testing.T) {
    tests := []struct {
        in   string
        want bool
    }{
        {"racecar", true},
        {"RaceCar", true},
        {"A man, a plan, a canal: Panama", true},
        {"not one", false},
    }
    for _, tc := range tests {
        t.Run(tc.in, func(t *testing.T) {
            if got := testingIsPalindromeASCII(tc.in); got != tc.want {
                t.Fatalf("got %v; want %v", got, tc.want)
            }
        })
    }
}

// EXERCISE 3: Implement testingSplitCSVSimple.
// Split by commas, trim spaces, allow empty fields.
// "a, b,,c" => ["a","b","","c"]
func TestTestingSplitCSVSimple(t *testing.T) {
    in := "a, b,,c"
```

```
    want := []string{"a", "b", "", "c"}
    got := testingSplitCSVSimple(in)
    if len(got) != len(want) {
        t.Fatalf("len=%d; want %d, got=%v", len(got), len(want), got)
    }
    for i := range want {
        if got[i] != want[i] {
            t.Fatalf("i=%d got %q want %q (got=%v)", i, got[i], want[i], got)
        }
    }
}

func testingNormalizeWhitespace(s string) string {
    // TODO: implement (hint: strings.Fields can help)
    _ = strings.Builder{}
    return ""
}

func testingIsPalindromeASCII(s string) bool {
    // TODO: implement
    return false
}

func testingSplitCSVSimple(s string) []string {
    // TODO: implement (hint: strings.Split + strings.TrimSpace)
    return nil
}
```

Tooling

Concise explanation: Go ships with a cohesive toolchain: `go test`, `go fmt` / `gofmt`, `go vet` (static checks), `go doc`, and `go fix` to modernize code. Formatting is culturally enforced and documented by the Go blog and `gofmt` docs; the command documentation lists official tooling entry points. [75]

.NET parallel: `dotnet format` and analyzers exist, but Go's ecosystem treats formatting + baseline tooling as "part of the language contract" in practice. [76]

Side-by-side (common commands)

```
# .NET
dotnet build
dotnet test
dotnet restore
```

```
# Go
go build ./...
go test ./...
go test -race ./...
go fmt ./...
go vet ./...
```

Unit-test exercises (copy-paste into `tooling_exercises_test.go`)

```
package learn

import (
    "bytes"
    "go/format"
    "testing"
)

// These exercises simulate "tooling thinking" via code,
// but in real life you'd actually run gofmt/go test/go vet.
// (gofmt itself is documented as the canonical formatter.)

// EXERCISE 1: Implement toolingFormatGoSource.
// Should return gofmt-formatted version of src using go/format.
func TestToolingFormatGoSource(t *testing.T) {
    in := "package p\n\nfunc  f(){return}\n"
    out, err := toolingFormatGoSource(in)
    if err != nil {
        t.Fatalf("unexpected err: %v", err)
    }
    // gofmt will normalize spacing; we just check it includes "func f()".
    if !bytes.Contains([]byte(out), []byte("func f()")) {
        t.Fatalf("formatted output missing expected substring; got:\n%s", out)
    }
}

// EXERCISE 2: Implement toolingLooksGofmted.
// Return true if formatting src with toolingFormatGoSource yields identical
bytes.
func TestToolingLooksGofmted(t *testing.T) {
    src := "package p\n\nfunc f() {}\n"
    if !toolingLooksGofmted(src) {
        t.Fatalf("expected true for already formatted source")
    }
    if toolingLooksGofmted("package p\n\nfunc  f(){}\n") {
        t.Fatalf("expected false for unformatted source")
    }
```

```
}

// EXERCISE 3: Implement toolingCountTestFunctions.
// Count occurrences of "func TestXxx(" in a Go file (simple string scan is
fine).
func TestToolingCountTestFunctions(t *testing.T) {
    src := `
package p
import "testing"
func TestA(t *testing.T) {}
func TestB(t *testing.T) {}
func helper() {}
`
    if got := toolingCountTestFunctions(src); got != 2 {
        t.Fatalf("got %d; want 2", got)
    }
}

func toolingFormatGoSource(src string) (string, error) {
    // TODO: implement (hint: format.Source)
    _, _ = format.Source([]byte("package p"))
    return "", nil
}

func toolingLooksGofmted(src string) bool {
    // TODO: implement
    return false
}

func toolingCountTestFunctions(src string) int {
    // TODO: implement
    return 0
}
```

Idiomatic patterns

Concise explanation: “Idiomatic Go” is less about clever tricks and more about predictable structure: useful zero values, small interfaces, explicit error handling with wrapping, channel-based orchestration when it clarifies ownership, and standardized formatting. The official `Effective Go` doc is still valuable for style fundamentals (with an explicit note that it doesn’t cover newer ecosystem changes deeply), and the Go wiki’s Code Review Comments is a living list of common review feedback. [77]

.NET parallel: A lot of “best practices” in C# are framework-driven. In Go, many best practices are language/tooling-driven (formatting, errors, package boundaries). [78]

Side-by-side (resource cleanup)

```
using var stream = File.OpenRead(path);
// use stream
```

```
f, err := os.Open(path)
if err != nil { return err }
defer f.Close()
// use f
```

Unit-test exercises (copy-paste into `idioms_exercises_test.go`)

```
package learn

import (
    "errors"
    "io"
    "strings"
    "testing"
)

type idiomsStringSet map[string]struct{}

// EXERCISE 1: Implement methods on idiomsStringSet so the zero value is useful.
// - Add(s string) should work even if the receiver is nil (initialize as
needed).
// - Has(s string) bool
// - Len() int
func TestIdiomsStringSetZeroValueUsable(t *testing.T) {
    var s idiomsStringSet // nil map
    if s.Len() != 0 || s.Has("x") {
        t.Fatalf("unexpected initial state")
    }
    s.Add("x")
    s.Add("y")
    if !s.Has("x") || s.Len() != 2 {
        t.Fatalf("set not behaving as expected: %#v", s)
    }
}

// EXERCISE 2: Implement idiomsReadAllAndClose.
// Must read everything and always close.
// If readErr and closeErr are both non-nil, return errors.Join(readErr,
closeErr).
func TestIdiomsReadAllAndClose(t *testing.T) {
    rc := io.NopCloser(strings.NewReader("hi"))
```

```
    b, err := idiomsReadAllAndClose(rc)
    if err != nil {
        t.Fatalf("unexpected err: %v", err)
    }
    if string(b) != "hi" {
        t.Fatalf("got %q", string(b))
    }
}

// EXERCISE 3: Implement idiomsPreferConcreteReturn.
// Accept io.Reader (interface), but return concrete []string.
// Split lines by '\n', ignoring empty trailing line.
func TestIdiomsPreferConcreteReturn(t *testing.T) {
    in := strings.NewReader("a\nb\n")
    got, err := idiomsPreferConcreteReturn(in)
    if err != nil {
        t.Fatalf("unexpected: %v", err)
    }
    if len(got) != 2 || got[0] != "a" || got[1] != "b" {
        t.Fatalf("got %#v", got)
    }
}

// TODO: implement these:

func (s idiomsStringSet) Add(v string) {
    // TODO: implement
}

func (s idiomsStringSet) Has(v string) bool {
    // TODO: implement
    return false
}

func (s idiomsStringSet) Len() int {
    // TODO: implement
    return 0
}

func idiomsReadAllAndClose(rc io.ReadCloser) ([]byte, error) {
    // TODO: implement
    _ = errors.Join(nil, nil)
    return nil, nil
}

func idiomsPreferConcreteReturn(r io.Reader) ([]string, error) {
    // TODO: implement
```

```
    return nil, nil
}
```

Small capstone project with tests integrating multiple concepts

This capstone is intentionally "small but real": it uses **modules/packages**, **interfaces**, **methods**, **explicit error handling**, **goroutines + channels**, **context cancellation**, and **table-driven tests**. Layout guidance is aligned with official "organizing a module" recommendations (including using `internal/` for non-exported implementation details). [79]

Design overview

Goal: implement a small `runner` library that executes tasks concurrently with a worker limit and returns a single aggregated error (`errors.Join`) while respecting cancellation via `context.Context`. This mirrors real production Go patterns: worker pools, cancellation, and explicit error aggregation. [80]

Module/package relationship diagram

```
graph TD
  A[cmd/demo] --> B[runner]
  B --> C[internal/runnerpool]
  C --> D[context]
  C --> E[errors]
  C --> F[sync]
```

File layout

Use a directory like:

- `go-capstone/`
- `go.mod`
- `runner/runner.go`
- `runner/runner_test.go`

This is a "multiple packages" module where `runner` is an importable package. The official layout docs show this pattern and explain how `go.mod`'s module path relates to imports. [81]

Code

`go.mod`

```
module example.com/go-capstone

go 1.22
```

`runner/runner.go`

```
package runner

import (
    "context"
    "errors"
    "fmt"
    "sync"
)

type Task func(context.Context) error

type Runner struct {
    Workers int
}

var ErrInvalidWorkers = errors.New("workers must be > 0")

type result struct {
    i   int
    err error
}

// Run executes tasks with a worker limit.
// If ctx is canceled, it stops scheduling new tasks and returns ctx.Err()
// joined with any task errors already produced.
func (r Runner) Run(ctx context.Context, tasks []Task) error {
    if r.Workers <= 0 {
        return ErrInvalidWorkers
    }
    if len(tasks) == 0 {
        return nil
    }

    jobs := make(chan int)
    results := make(chan result)

    var wg sync.WaitGroup
    for w := 0; w < r.Workers; w++ {
        wg.Add(1)
        go func() {
```

```
            defer wg.Done()
            for i := range jobs {
                err := tasks[i](ctx)
                results <- result{i: i, err: err}
            }
        }()
    }

    // Close results after all workers exit.
    go func() {
        wg.Wait()
        close(results)
    }()

    // Feed jobs; stop early on cancellation.
    stoppedEarly := false
    go func() {
        defer close(jobs)
        for i := range tasks {
            select {
            case <-ctx.Done():
                stoppedEarly = true
                return
            case jobs <- i:
            }
        }
    }()

    errs := make([]error, len(tasks))
    for res := range results {
        errs[res.i] = res.err
    }

    combined := errors.Join(errs...)
    if stoppedEarly && ctx.Err() != nil {
        return errors.Join(ctx.Err(), combined)
    }
    return combined
}

func (r Runner) String() string {
    return fmt.Sprintf("Runner(workers=%d)", r.Workers)
}
```

`runner/runner_test.go`

```
package runner

import (
    "context"
    "errors"
    "sync/atomic"
    "testing"
    "time"
)

func TestRunner_RunValidatesWorkers(t *testing.T) {
    r := Runner{Workers: 0}
    if err := r.Run(context.Background(), nil); !errors.Is(err,
ErrInvalidWorkers) {
        t.Fatalf("got %v; want ErrInvalidWorkers", err)
    }
}

func TestRunner_RunAggregatesErrors(t *testing.T) {
    e1 := errors.New("e1")
    e2 := errors.New("e2")

    tasks := []Task{
        func(context.Context) error { return nil },
        func(context.Context) error { return e1 },
        func(context.Context) error { return e2 },
    }

    r := Runner{Workers: 2}
    err := r.Run(context.Background(), tasks)
    if err == nil {
        t.Fatalf("expected non-nil")
    }
    if !errors.Is(err, e1) || !errors.Is(err, e2) {

t.Fatalf("expected joined error to match component errors via errors.Is; got
%v", err)
    }
}

func TestRunner_RunStopsSchedulingOnCancel(t *testing.T) {
    var started int32

    block := make(chan struct{})
    defer close(block)

    tasks := make([]Task, 100)
```

```
    for i := range tasks {
        tasks[i] = func(ctx context.Context) error {
            atomic.AddInt32(&started, 1)
            select {
            case <-ctx.Done():
                return ctx.Err()
            case <-block:
                return nil
            }
        }
    }

    ctx, cancel := context.WithCancel(context.Background())
    cancel()

    r := Runner{Workers: 4}
    _ = r.Run(ctx, tasks)

    // We canceled before scheduling, so "started" should be small (possibly 0),
    // but depending on timing it could be >0 if some tasks got scheduled early.
    // We keep the assertion conservative: it should NOT say "100".
    if atomic.LoadInt32(&started) >= 100 {
        t.Fatalf("unexpected: started=%d", started)
    }
}

func TestRunner_RunZeroTasks(t *testing.T) {
    r := Runner{Workers: 1}
    if err := r.Run(context.Background(), nil); err != nil {
        t.Fatalf("got %v; want nil", err)
    }
}

func TestRunner_String(t *testing.T) {
    r := Runner{Workers: 3}
    if got := r.String(); got != "Runner(workers=3)" {
        t.Fatalf("got %q", got)
    }
}

func TestRunner_RunDoesNotHang(t *testing.T) {
    // Sanity test: ensure no deadlock via timeout.
    ctx, cancel := context.WithTimeout(context.Background(),
200*time.Millisecond)
    defer cancel()

    tasks := []Task{
        func(context.Context) error { return nil },
```

```
        func(context.Context) error { return nil },
    }

    r := Runner{Workers: 2}
    _ = r.Run(ctx, tasks)
}
```

Why this capstone matches real Go practice

- Worker pool pattern and channel orchestration are canonical Go concurrency talk tracks and show up in official material (pipelines, context, concurrency in the Tour). [82]
- Cancellation and deadlines are standardized via `context.Context`, which is designed for propagation across goroutine boundaries. [17]
- Multi-error aggregation is now standardized with `errors.Join`, and inspection happens via `errors.Is/As`. [36]
- This structure aligns with official module layout guidance, and the tests follow the `testing` package model (`TestXxx`). [83]

References and reading list

These are the highest-leverage sources used in this report (prioritizing Go official docs + Go blog posts, plus Go by Example where requested):

- Go installation and verification docs. [84]
- Go tutorial: getting started (install → hello world → `go` command). [85]
- Go tutorial: create a module (introduces error handling, slices/maps, unit testing). [86]
- The Go language specification (authoritative reference for syntax/semantics). [22]
- A Tour of Go (interactive exercises; includes concurrency, interfaces, errors). [28]
- Effective Go (idioms/style fundamentals; includes note about age/coverage). [52]
- Go blog: “Errors are values” (philosophy of explicit errors). [35]
- Go blog: “Error handling and Go” (practical error patterns). [34]
- `errors` package docs (`Is/As/Unwrap/Join`, wrapping tree model). [36]
- Go blog: “Defer, Panic, and Recover” (semantics + best-use cases). [37]
- Tour: channels + select (mechanics). [87]
- Go blog: pipelines and cancellation (concurrency pattern). [39]
- Go blog: context (propagate cancellation/deadlines). [40]
- Go memory model (what synchronization guarantees mean). [41]
- Race detector docs + blog (how to use, why it matters). [88]
- Go modules reference + go.mod reference + modules blog series. [89]
- How to Write Go Code (import path model; practical structure basics). [43]
- Organizing a Go module (official layout guidance: `internal/`, `cmd/`). [46]
- Go blog: `gofmt` (canonical formatting) + gofmt docs. [90]
- Command documentation (`go fmt`, `go doc`, `go vet`, etc.). [50]
- Go Wiki: Code Review Comments (common idioms/pitfalls). [32]
- Go Wiki: Table-driven tests (idiomatic testing style). [48]
- Go by Example: testing (requested resource). [49]

- Go 1.18 release notes + generics intro + “When to use generics” + type inference updates (for generics status). [91]
- C# exception handling reference (for the .NET comparison). [92]
- .NET TPL and task-based async programming docs (for concurrency comparison). [93]
- NuGet restore docs (for package management comparison). [94]

---

[1] [54] [85] Tutorial: Get started with Go
https://go.dev/doc/tutorial/getting-started?utm_source=chatgpt.com

[2] [20] [55] [84] Download and install
https://go.dev/doc/install?utm_source=chatgpt.com

[3] [12] [22] [57] [58] [59] [61] [63] The Go Programming Language Specification
https://go.dev/ref/spec?utm_source=chatgpt.com

[4] [64] [69] [92] Exception-handling statements - throw and try, catch, finally
https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/exception-handling-statements?utm_source=chatgpt.com

[5] [60] Generic classes and methods - C#
https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/generics?utm_source=chatgpt.com

[6] [93] Task Parallel Library (TPL) - .NET
https://learn.microsoft.com/en-us/dotnet/standard/parallel-programming/task-parallel-library-tpl?utm_source=chatgpt.com

[7] [56] [94] dotnet restore command - .NET CLI
https://learn.microsoft.com/en-us/dotnet/core/tools/dotnet-restore?utm_source=chatgpt.com

[8] [47] [74] testing package
https://pkg.go.dev/testing?utm_source=chatgpt.com

[9] [24] [76] [78] [90] go fmt your code
https://go.dev/blog/gofmt?utm_source=chatgpt.com

[10] [43] [72] [73] How to Write Go Code
https://go.dev/doc/code?utm_source=chatgpt.com

[11] [45] go.mod file reference
https://go.dev/doc/modules/gomod-ref?utm_source=chatgpt.com

[13] [31] [66] [67] Interfaces
https://go.dev/tour/methods/9?utm_source=chatgpt.com

[14] [37] Defer, Panic, and Recover
https://go.dev/blog/defer-panic-and-recover?utm_source=chatgpt.com

[15] [25] [38] [70] Welcome to a tour of Go
https://go.dev/tour/list?utm_source=chatgpt.com

[16] [87] Channels
https://go.dev/tour/concurrency/2?utm_source=chatgpt.com

[17] context package
https://pkg.go.dev/context?utm_source=chatgpt.com

18 36 errors package - errors - Go Packages
https://pkg.go.dev/errors

19 Using Go Modules
https://go.dev/blog/using-go-modules?utm_source=chatgpt.com

21 86 Tutorial: Create a Go module
https://go.dev/doc/tutorial/create-module?utm_source=chatgpt.com

23 28 A Tour of Go
https://go.dev/tour/?utm_source=chatgpt.com

26 Go maps in action
https://go.dev/blog/maps?utm_source=chatgpt.com

27 52 77 Effective Go
https://go.dev/doc/effective_go?utm_source=chatgpt.com

29 65 Methods
https://go.dev/tour/methods?utm_source=chatgpt.com

30 Go Wiki: MethodSets
https://go.dev/wiki/MethodSets?utm_source=chatgpt.com

32 Go Wiki: Go Code Review Comments
https://go.dev/wiki/CodeReviewComments?utm_source=chatgpt.com

33 68 Errors
https://go.dev/tour/methods/19?utm_source=chatgpt.com

34 Error handling and Go
https://go.dev/blog/error-handling-and-go?utm_source=chatgpt.com

35 Errors are values
https://go.dev/blog/errors-are-values?utm_source=chatgpt.com

39 80 82 Go Concurrency Patterns: Pipelines and cancellation
https://go.dev/blog/pipelines?utm_source=chatgpt.com

40 Go Concurrency Patterns: Context
https://go.dev/blog/context?utm_source=chatgpt.com

41 The Go Memory Model
https://go.dev/ref/mem?utm_source=chatgpt.com

42 88 Data Race Detector
https://go.dev/doc/articles/race_detector?utm_source=chatgpt.com

44 89 Go Modules Reference
https://go.dev/ref/mod?utm_source=chatgpt.com

46 79 81 83 Organizing a Go module - The Go Programming Language
https://go.dev/doc/modules/layout

48 Go Wiki: TableDrivenTests
https://go.dev/wiki/TableDrivenTests?utm_source=chatgpt.com

49 Go by Example: Testing
https://gobyexample.com/testing?utm_source=chatgpt.com

50 Command Documentation
https://go.dev/doc/cmd?utm_source=chatgpt.com

51 Using go fix to modernize Go code
https://go.dev/blog/gofix?utm_source=chatgpt.com

53 Share Memory By Communicating
https://go.dev/blog/codelab-share?utm_source=chatgpt.com

62 Select
https://go.dev/tour/concurrency/5?utm_source=chatgpt.com

71 Task-based asynchronous programming - .NET
https://learn.microsoft.com/en-us/dotnet/standard/parallel-programming/task-based-asynchronous-programming?utm_source=chatgpt.com

75 go command - cmd/go
https://pkg.go.dev/cmd/go?utm_source=chatgpt.com

91 Go 1.18 Release Notes
https://go.dev/doc/go1.18?utm_source=chatgpt.com